

Term Project IML

Group 33: Dmitrii Gorovoi (Kaggle: GorovoiD)

Introduction

The goal of the NPF Challenge is to classify events detected in the atmosphere using data provided. The events are categorized into four classes:

- “nonevent”,
- “Ia”,
- “Ib”,
- “II”.

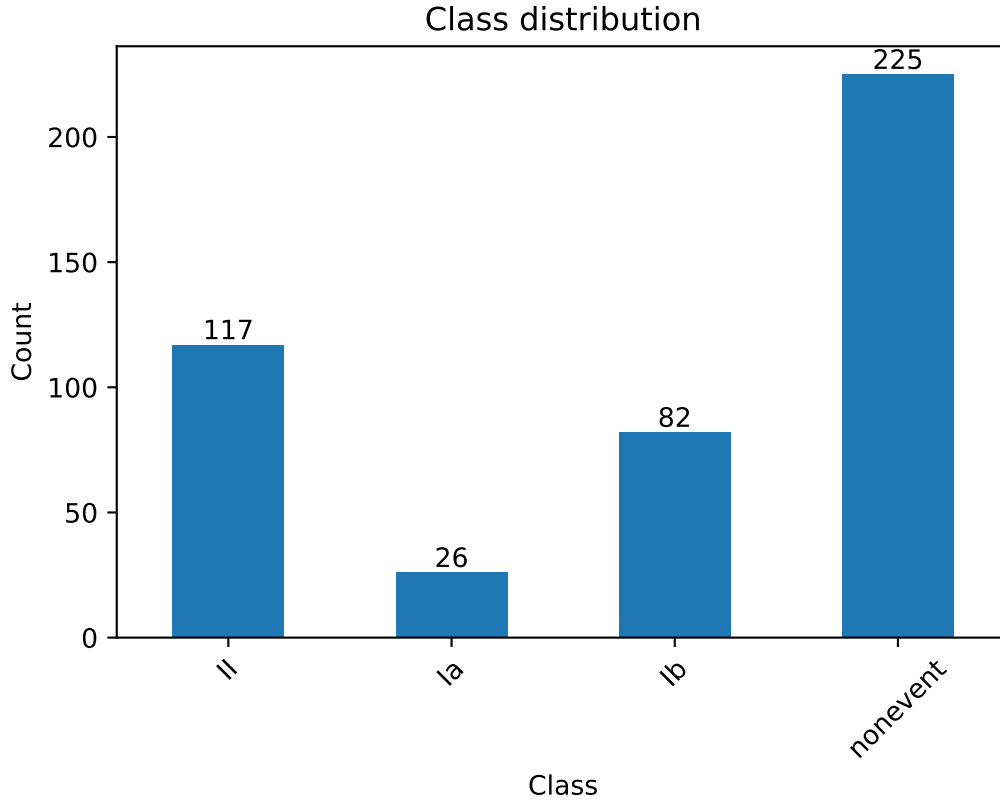
While “Ia”, “Ib”, and “II” represent different types of occurrence of NPF, “nonevent” indicates the absence of NPF event. This is a multi-class classification problem where we need to assign each observation to one of the four classes based on the features provided in the dataset.

We are required to produce in general 3 outputs: “nonevent” vs “event”, type of an event, and probability of being event (non-event) occurring.

My first approach was to treat this as a multi-class classification problem with 4 classes: “nonevent”, “Ia”, “Ib”, “II”, however as it will be shown it did not yield good results. Therefore I decided to treat this as 2-step problem: first classify “nonevent” vs “event” and then classify the event class into “Ia”, “Ib”, “II”.

Data Exploration

The initial look at the data shows that there are 50 variables where each has mean and std values therefore there are 100 features in total.



As can be seen from the bar plot, the dataset is imbalanced with “nonevent” having the highest number of samples and “Ia” having the lowest. This means that some algorithms can be biased towards the majority class because 26 vs 225 is a significant difference.

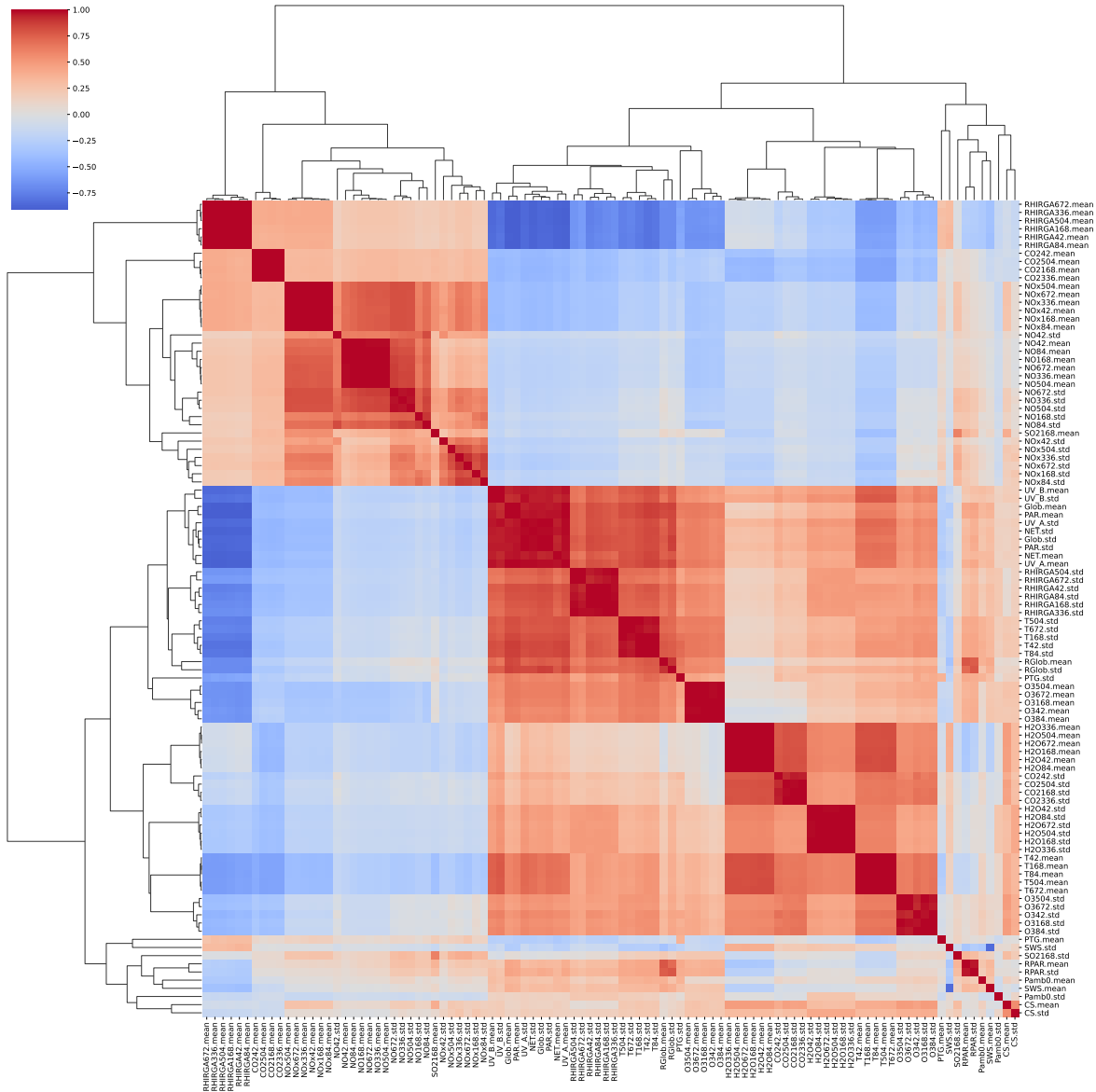
I decided to use ANOVA F-test to evaluate the importance of each feature. While this method has a limitation of assuming the independence of features, it is a good starting point for feature selection in classification problems. Here are the top 10 features based on the ANOVA F-test:

Top 10 features based on ANOVA F-test:

RHIRGA84.mean	F-score: 131.3810	p-value: 5.4324e-61
RHIRGA168.mean	F-score: 130.6881	p-value: 9.4251e-61
RHIRGA336.mean	F-score: 129.9970	p-value: 1.6352e-60
RHIRGA504.mean	F-score: 129.4898	p-value: 2.4522e-60
RHIRGA42.mean	F-score: 128.9464	p-value: 3.7886e-60
RHIRGA672.mean	F-score: 128.9255	p-value: 3.8526e-60

Glob.mean F-score: 105.4050 p-value: 1.3679e-51
 PAR.mean F-score: 98.7472 p-value: 5.0051e-49
 RHIRGA42.std F-score: 86.6991 p-value: 3.2884e-44
 UV_A.mean F-score: 84.9405 p-value: 1.7400e-43

I used seaborn's clustermap to visualize the correlation matrix of the features as well as to organize them in clusters based on their similarity.



Later on, I might consider removing some of the highly correlated features to reduce redundancy

and improve model performance.

Data Preprocessing

The next step before training the model is to preprocess the data.

Feature Selection

Based on the clustermap we can see some very dense blocks on the main diagonal. I decided to choose clusters based on these main diagonal blocks and select one feature from each cluster based on the highest ANOVA F-score. This way I can reduce the number of features while retaining the most important information.

Such clustermap can be explained by that fact that some particles are very similar in nature and therefore their measurements are highly correlated. For example, `RHIRGA672.mean` and `RHIRGA336.mean` are both measurements related to the RHIRGA particle, but at different wavelengths (672 nm and 336 nm respectively). Since they are measuring similar properties of the same particle type, it is expected that their measurements would be correlated.

Therefore reducing number of features would not lead to significant loss of information while also reducing the risk of overfitting and improving model interpretability.

Selected 36 features from 100 original features.

36 is the number of bright red clusters (very high correlation) on the clustermap main diagonal.

Standardization

Features have different scales which can affect the performance of some machine learning algorithms. Therefore, it is important to normalize the features before training a model. For the models which require standardized data (Logistic Regression, SVM, kNN) I used `sklearn`'s `StandardScaler` to standardize the features to have mean 0 and standard deviation 1."

Model Training and Evaluation

Firstly I implemented the NFP Aggregate Scorer as described in the project instructions.

```
def kaggle_scorer(estimator, X, y_true):
    y_pred = estimator.predict(X)
    y_proba = estimator.predict_proba(X)

    class_idx = {c: i for i, c in enumerate(estimator.classes_)}
    p_pred = np.array([
        y_proba[i, class_idx[y_pred[i]]]
        for i in range(len(y_pred))
    ])

    solution = pd.DataFrame({
        "class4": y_true.values
    })

    submission = pd.DataFrame({
        "class4": y_pred,
        "p": p_pred
    })

    return kaggle_score(solution, submission, row_id_column_name="id")
```

This score will be used to evaluate the model performance during training and hyperparameter tuning. Because it is the same score used in the competition.

Multi-class Classification

I decided to start with random forest classifier as it is a robust and versatile algorithm that can handle multi-class classification problems well.

```
Binary Accuracy = 0.87778
Perplexity = 6.20001
Multi-Class Accuracy = 0.71111
Combined Score = 0.52963
NFP Aggregate Score on test set: 0.5296
```

As we can see the initial score is not so good. Therefore I used cross-validation to get a better estimate of the model performance. I used **StratifiedKfold** with 5 splits to ensure that each

fold has a similar class distribution as the whole dataset. This was the parameter distribution tested:

```
Fitting 5 folds for each of 30 candidates, totalling 150 fits
Best score: 0.5103703703703704
Best parameters:
n_estimators: 300
min_samples_split: 5
min_samples_leaf: 1
max_features: log2
max_depth: None
```

Overall 0.51 is not a good score for this competition. One of the areas of improvement is to try different algorithms. I decided to try Logistic Regression as it is a simple yet effective algorithm for multi-class classification problems.

```
Fitting 5 folds for each of 6 candidates, totalling 30 fits
Best CV score: 0.4962962962962963
Best params: {'C': 0.1}
```

The logistic regression model achieved a CV score of approximately 0.49 which is even lower than the random forest model. So I decided that this approach is not effective in general.

2-step Classification

The new approach is to first classify “nonevent” vs “event” and then classify the event class into “Ia”, “Ib”, “II”. Firstly I decided to use the logistic regression for the binary classification step and for the multi-class classification step, I decided to use the random forest classifier. I also split the training data into training and validation sets to evaluate the performance of the models. During the training the same selected features and standardized (for LogReg) data was used.

I used this setup for Random Forest model training:

```
bin_model = LogisticRegression(
    max_iter=5000,
    class_weight="balanced",
    solver="lbfgs",
    n_jobs=-1
)
```

```

rf = RandomForestClassifier(
    n_estimators=500,
    max_depth=None,
    class_weight="balanced_subsample",
    random_state=42,
    n_jobs=-1
)

```

Where `bin_model` is trained to classify “nonevent” vs “event” and `rf` is trained to classify the event class into “Ia”, “Ib”, “II”.

```

Binary Accuracy = 0.91333
Perplexity = 1.26789
Multi-Class Accuracy = 0.91333
Combined Score = 0.85293
score: 0.8529270359697128

```

This approach yielded a score of approximately 0.73 in kaggle which is a significant improvement over the previous methods. The main idea behind this approach is to break down the complex multi-class classification problem into two simpler problems which can be solved more effectively. There is approximately similar number of “event” and “nonevent” samples in the dataset which makes the binary classification easier. Once the “event” samples are identified, the multi-class classification can be performed on a smaller subset of data which can lead to better performance because the model can focus on distinguishing between the different event classes without being distracted by the “nonevent” class.

Furthermore, by using cross-validation and hyperparameter tuning, I decided to optimize both models used in this 2-step approach to improve the overall performance.

Logistic Regression

Tuning the Logistic Regression model for the binary classification step:

```

logreg_param_dist = {
    "logreg__C": [0.001, 0.01, 0.1, 1, 10, 100]
}

```

```

Fitting 5 folds for each of 6 candidates, totalling 30 fits
Best LogReg params: {'logreg__C': 1}

```

Random Forest

Tuning the Random Forest model for the multi-class classification step:

```
rf_param_dist = {  
    "n_estimators": [300, 500, 800],  
    "max_depth": [None, 10, 20, 30],  
    "min_samples_leaf": [1, 3, 5],  
    "max_features": ["sqrt", "log2", 0.5]  
}
```

Fitting 5 folds for each of 15 candidates, totalling 75 fits

Best RF params:

n_estimators: 800

min_samples_leaf: 5

max_features: sqrt

max_depth: 20

Binary Accuracy = 0.91333

Perplexity = 1.26789

Multi-Class Accuracy = 0.89111

Combined Score = 0.84552

Final 2-stage model score: 0.8455196285623053

The CV approach resulted in 0.77 in public score however the private score was lower at 0.70. This indicates some overfitting to the public leaderboard data.

I decided to test other algorithms for the multi-class classification step such as SVM and kNN to see if they can yield better results.

SVM

I used similar setup for SVM model training as in the Random Forest case:

```
bin_model = Pipeline([  
    ("scaler", StandardScaler()),  
    ("logreg", LogisticRegression(  
        max_iter=5000,  
        class_weight="balanced",  
        solver="lbfgs",  
        n_jobs=-1  
    ))  
])
```



```

])

svm_model = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", LinearSVC(
        C=0.02,
        class_weight="balanced",
        random_state=42
    ))
])

```

The SVM model achieves a score of 0.726 in kaggle which is slightly lower than the random forest model. So I turned to kNN as the final try.

kNN

For kNN I started with Cross-Validation and hyperparameter tuning straight from the beginning to find best k and other parameters:

```

param_dist = {
    "knn__n_neighbors": list(range(1, 51, 2)),
    "knn__weights": ["uniform", "distance"],
    "knn__p": [1, 2],
    "knn__leaf_size": [10, 20, 30, 40, 60]
}

```

The model setup was similar to previous cases. With `bin_model` for binary classification and `kNN` for multi-class classification.

```

Best kNN params:
knn__weights: distance
knn__p: 2
knn__n_neighbors: 31
knn__leaf_size: 10

```

The kNN model achieved a score of 0.731 is the second best score among the models tested.

Conclusion

Overall we can see that the idea of 2-step approach for solving this problem turns out much more effective than trying to solve the multi-class classification problem directly. By breaking down the problem into simpler sub-problems, we can achieve better performance and more accurate predictions.

The direct approach didn't yield scores better than 0.51 while the 2-step approach achieved scores up to 0.73594 in kaggle. This result was achieved by using Logistic Regression for the binary classification step and Random Forest for the multi-class classification step.

Changes in the models used in each step and use of hyperparameter tuning with cross-validation also helped to improve the performance further, however the improvements had really small margins.

Self-assessment

Overall different approaches were tried and the best one was selected based on empirical results. The code is well documented and organized into sections for better readability. The models were evaluated using appropriate metrics and cross-validation techniques to ensure robustness. Hyperparameter tuning was performed to optimize model performance.

Understanding of the specifications of the problem was shown in the feature selection section where correlated features were reduced to avoid redundancy.

The final score achieved in kaggle was 0.73594 which is a good result for this competition. However, there is still room for improvement as the best score was 0.78.

All the deliverables were submitted on time.

I would rate this project as 5 or 4.